

Recurrent Neural Networks

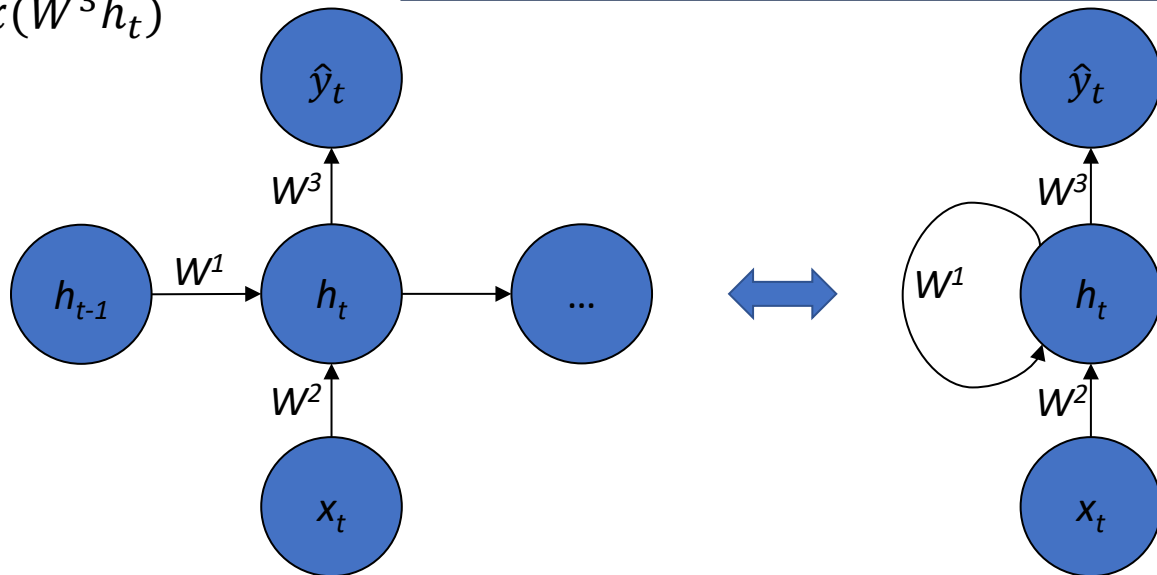
Recurrent Neural Networks (RNNs)

Why TanH?

- At a single time step

- $h_t = \tanh(W^1 h_{t-1} + W^2 x_t)$
- $\hat{y}_t = \text{softmax}(W^3 h_t)$

1. Tanh的导数范围在(0, 1]范围, Sigmoid导数最大0.25, 减缓梯度消失与爆炸
2. TanH输出为0中心, 减缓优化震荡
3. 饱和区间具有一定的稳定性, 类似记忆



Language Models

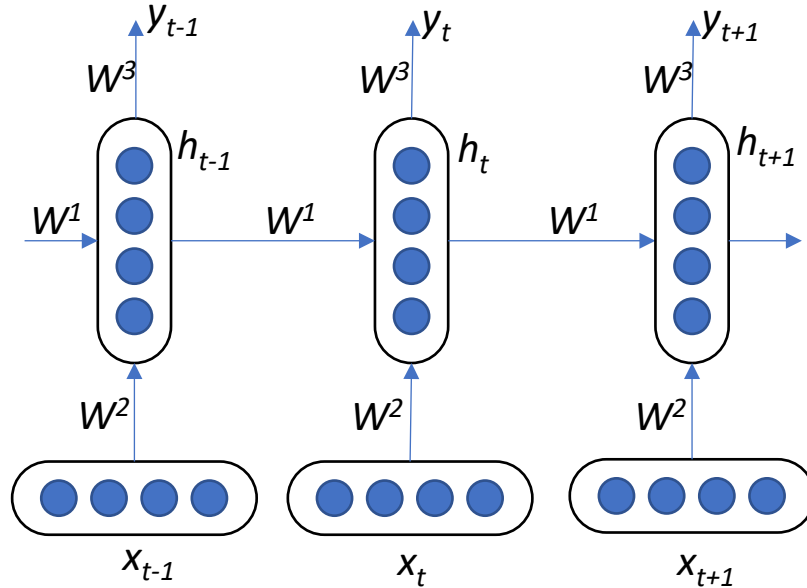
- A language model computes a probability for a sequence of word:
 $P(w_1, \dots w_n)$ or predicts a probability for the next word:
 $P(w_{n+1} | w_1, \dots w_n)$
- Useful for machine translation, speech recognition, and so on
 - Word ordering
 - $P(\text{the cat is small}) > P(\text{small the is cat})$
 - Word choice
 - $P(\text{there are } \mathbf{four} \text{ cats}) > P(\text{there are } \mathbf{for} \text{ cats})$

Traditional Language Models

- An incorrect but necessary **Markov** assumption!
 - Probability is usually conditioned on window of $n-1$ previous words
 - $P(w_1, \dots, w_m) = \prod_{i=1}^m P(w_i | w_1, \dots, w_{i-1}) \approx \prod_{i=1}^m P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$
- How to estimate probabilities
 - $p(w_2 | w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)}$ $p(w_3 | w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}$
- Performance improves with keeping around higher n-grams counts and doing **smoothing**, such as backoff (e.g. if 4-gram not found, try 3-gram, etc)
- Disadvantages
 - There are A LOT of n-grams!
 - Cannot see too long history
 - $P(\text{坐/作 了一整天的 火车/作业})$

Recurrent Neural Networks (RNNs)

- Condition the neural network on all previous inputs
- RAM requirement only scales with number of inputs



Recurrent Neural Network Language Model (RNNLM)

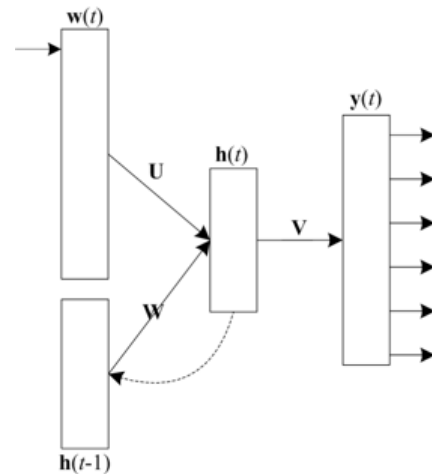
- Modeling the infinite context (Mikolov et al. 2010-2013)

- Compute:

$$\mathbf{h}(t) = f(\mathbf{U}\mathbf{w}(t) + \mathbf{W}\mathbf{h}(t-1))$$

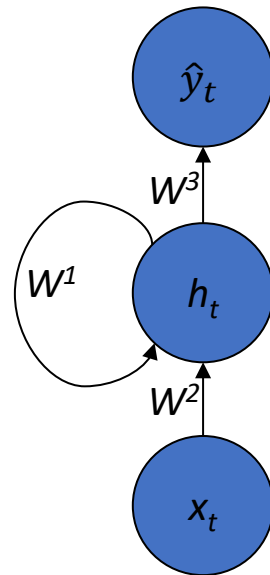
$$\mathbf{y}(t) = g(\mathbf{V}\mathbf{h}(t))$$

- $\mathbf{U}$ is the Embedding Matrix



BackPropagation Through Time (BPTT)

- Total error is the sum of each error at time step t
 - $\frac{\partial E}{\partial W} = \sum_{t=1}^T \frac{\partial E_t}{\partial W}$
- $\frac{\partial E_t}{\partial W^3} = \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial W^3}$ is easy to be calculated
- But to calculate $\frac{\partial E_t}{\partial W^1} = \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial W^1}$ is hard (also for W^2)
- Because $h_t = \tanh(W^1 h_{t-1} + W^2 x_t)$ depends on h_{t-1} , which depends on W^1 and h_{t-2} , and so on.



BackPropagation Through Time (BPTT)

- Use the same as the backpropagation algorithm as we use in deep feedforward NN, but summing up the gradients for W^1
- BPTT is just a fancy name for standard backpropagation on an unrolled RNN

- $$\frac{\partial E}{\partial W^1} = \sum_{t=1}^T \frac{\partial E_t}{\partial W^1}$$
- $$\frac{\partial E_t}{\partial W^1} = \sum_{k=1}^t \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W^1}$$

The vanishing gradient problem

- $\frac{\partial E_t}{\partial W^1} = \sum_{k=1}^t \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W^1}$, $h_t = \tanh(W^1 h_{t-1} + W^2 x_t)$
- $\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} = \prod_{j=k+1}^t W^1 \text{diag}[\tanh'(\dots)]$
- $\left\| \frac{\partial h_t}{\partial h_{t-1}} \right\| \leq \gamma \|W^1\| \leq \gamma \lambda_1$
 - where γ is bound $\|\text{diag}[\tanh'(\dots)]\|$, λ_1 is the largest singular value of W^1
- $\left\| \frac{\partial h_t}{\partial h_k} \right\| \leq (\gamma \lambda_1)^{t-k} \rightarrow 0$, if $\gamma \lambda_1 < 1$
- This can become very small or very large quickly $\rightarrow$ Vanishing or exploding gradient
 - Trick for exploding gradient: clipping trick (set a threshold)

A “solution”

- Intuition
 - Ensure $\gamma\lambda_1 \geq 1 \rightarrow$ to prevent vanishing gradients
- So ...
 - Proper initialization of the W
 - To use ReLU instead of tanh or sigmoid activation functions

A better “solution”

- Recall the original transition equation

- $h_t = \tanh(W^1 h_{t-1} + W^2 x_t)$

- We can instead update the state **additively**

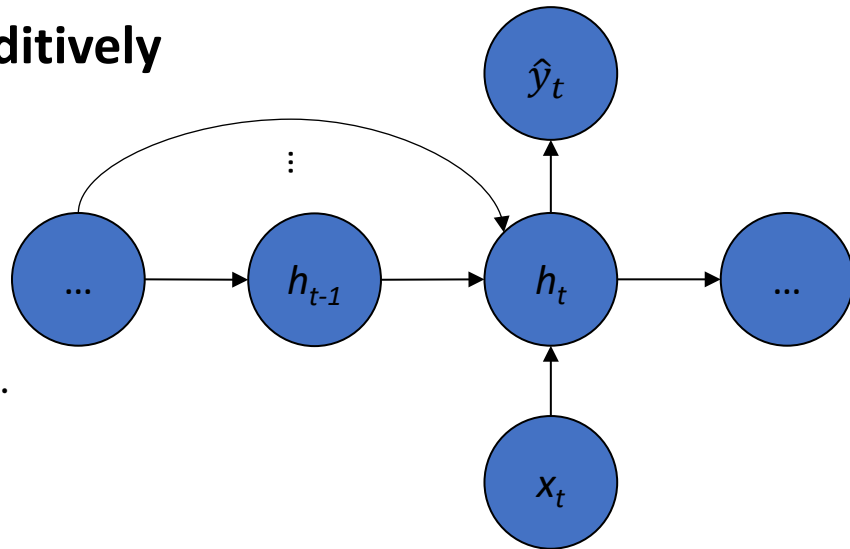
- $u_t = \tanh(W^1 h_{t-1} + W^2 x_t)$

- $h_t = h_{t-1} + u_t$

- then, $\left\| \frac{\partial h_t}{\partial h_{t-1}} \right\| = 1 + \left\| \frac{\partial u_t}{\partial h_{t-1}} \right\| \geq 1$

- On the other hand

- $h_t = h_{t-1} + u_t = h_{t-2} + u_{t-1} + u_t = \dots$



A better “solution” (cont.)

- Interpolate between old state and new state (“choosing to **forget**”)
 - $f_t = \sigma(W^f x_t + U^f h_{t-1})$
 - $h_t = f_t \odot h_{t-1} + (1 - f_t) \odot u_t$
- Introduce a separate **input gate** i_t
 - $i_t = \sigma(W^i x_t + U^i h_{t-1})$
 - $h_t = f_t \odot h_{t-1} + i_t \odot u_t$
- Selectively expose memory cell c_t with an **output gate** o_t
 - $o_t = \sigma(W^o x_t + U^o h_{t-1})$
 - $c_t = f_t \odot c_{t-1} + i_t \odot u_t$
 - $h_t = o_t \odot \tanh(c_t)$

Long Short-Term Memory (LSTM)

$$u_t = \tanh(W h_{t-1} + V x_t)$$

$$f_t = \text{sigmoid}(W_f h_{t-1} + V_f x_t)$$

$$i_t = \text{sigmoid}(W_i h_{t-1} + V_i x_t)$$

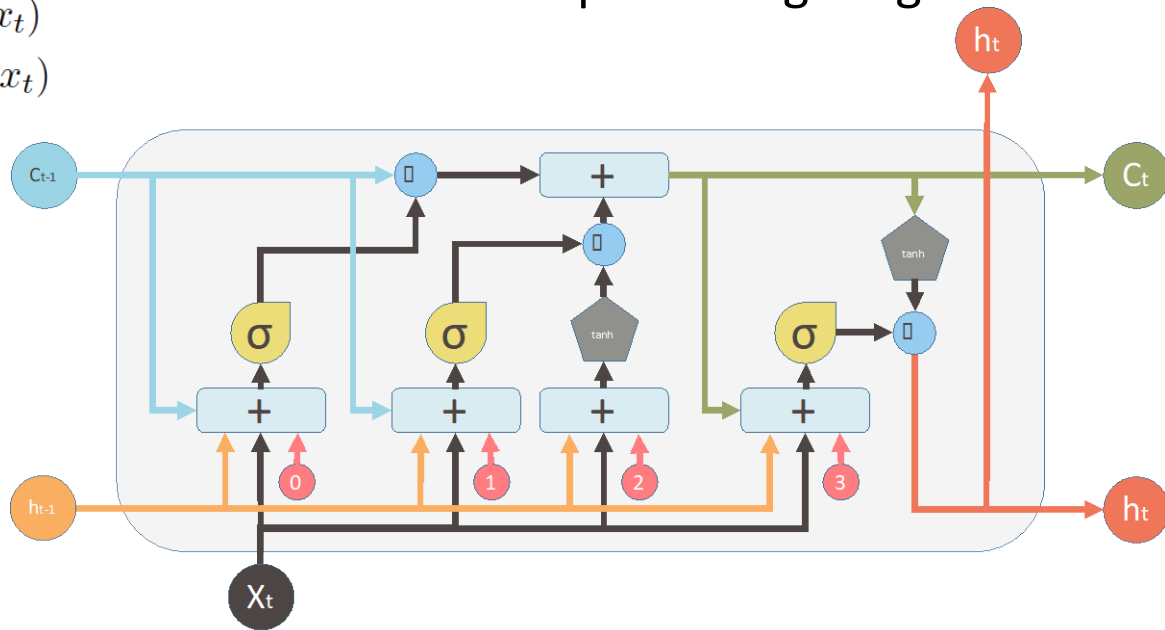
$$o_t = \text{sigmoid}(W_o h_{t-1} + V_o x_t)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot u_t$$

$$h_t = o_t \odot \tanh(c_t)$$

$$y_t = U h_t$$

- Hochreiter & Schmidhuber, 1997
- LSTM = additive updates + gating



Gated Recurrent Unites, GRU (Cho et al. 2014)

- Main ideas
 - Keep around memories to capture long distance dependencies
 - Allow error messages to flow at different strengths depending on the inputs
- Update gate
 - Based on current input and hidden state
 - $z_t = \sigma(W^z x_t + U^z h_{t-1})$
- Reset gate
 - Similarly but with different weights
 - $r_t = \sigma(W^r x_t + U^r h_{t-1})$

GRU

- New memory content

- $\tilde{h}_t = \tanh(Wx_t + r_t \odot Uh_{t-1})$

- Final memory at time step combines current and previous time steps

- $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}$

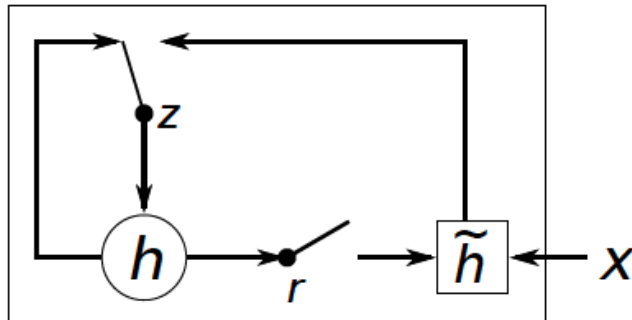
- Update gate z controls how much of past state should matter now

- If z closed to 1, then we can copy information in that unit through many time steps $\rightarrow$ less vanishing gradient!

- If reset gate r unit is close to 0, then this ignores previous memory and only stores the new input information $\rightarrow$ allows model to drop information that is irrelevant in the future

- Units with long term dependencies have active update gates z

- Units with short-term dependencies often have reset gates r very active

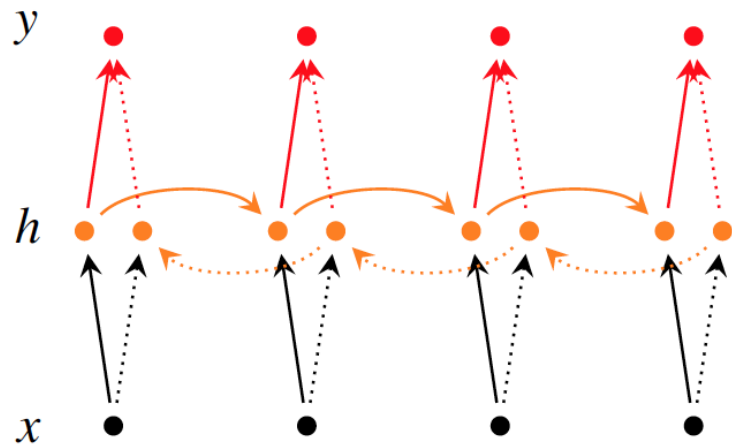


LSTM vs. GRU

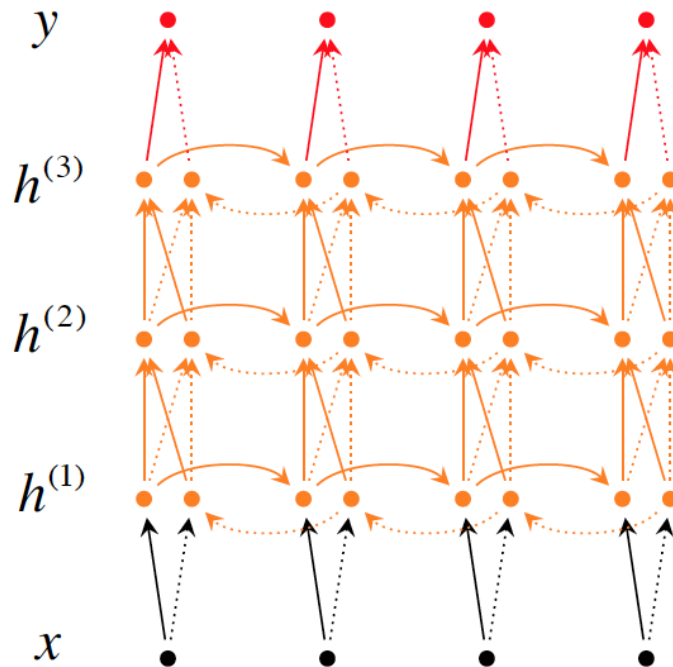
- No clear winner!
- Tuning hyperparameters like layer size is probably more important than picking the ideal architecture
- GRUs have fewer parameters and thus may train a bit faster or need less data to generalize
- If you have enough data, the greater expressive power of LSTMs may lead to better results.

More RNNs

- Bidirectional RNN

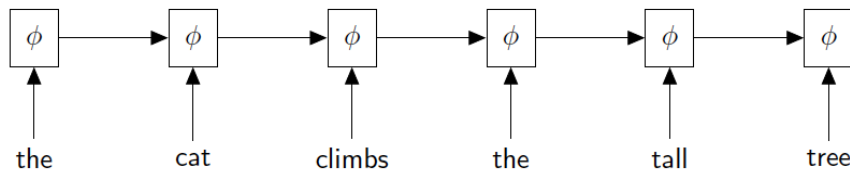


- Deep Bidirectional RNN



Tree-LSTMs

- Traditional Sequential Composition



- Tree-Structured Composition

